Page Contents

Introduction(⇘ #introduction)

Fundamental Keyboard Navigation Conventions(⇘ #fundamentalkeyboardnavigationconventions)

Discernible and Predictable Keyboard Focus(⇘ #discernibleandpredictablekeyboardfocus)

Focus VS Selection and the Perception of Dual Focus(⇘ #focusvsselectionandtheperceptionofdualfocus)

Deciding When to Make Selection Automatically Follow Focus(⇘ #decidingwhentomakeselectionautomaticallyfollowfocus)

Keyboard Navigation Between Components (The Tab Sequence)(⇘ #keyboardnavigationbetweencomponents(thetabsequence))

Keyboard Navigation Inside Components(⇘ #keyboardnavigationinsidecomponents)

Focusability of disabled controls(⇘ #focusabilityofdisabledcontrols)

Key Assignment Conventions for Common Functions(⇘ #keyassignmentconventionsforcommonfunctions)

Keyboard Shortcuts(⇘ #keyboardshortcuts)

Developing a Keyboard Interface

Introduction

Unlike native HTML form elements, browsers do not provide keyboard support for graphical user interface (GUI) components that are made accessible with ARIA; authors have to provide the keyboard support in their code. This section describes the principles and methods for making the functionality of a web page that includes ARIA widgets, such as menus and grids, as well as interactive components, such as toolbars and dialogs, operable with a keyboard. Along with the basics of focus management, this section offers guidance toward the objective of providing experiences to people who rely on a keyboard that are as efficient and enjoyable as the experiences available to others.

This section covers:

1. Understanding fundamental principles of focus movement conventions used in ARIA design patterns.
2. Maintaining visible focus, predictable focus movement, and distinguishing between keyboard focus and the selected state.
3. Managing movement of keyboard focus between components, e.g., how the focus moves when the `Tab` and `Shift` + `Tab` keys are pressed.

4. Managing movement of keyboard focus inside components that contain multiple focusable elements, e.g., two different methods for programmatically exposing focus inside widgets like radio groups, menus, listboxes, trees, and grids.
5. Determining when to make disabled interactive elements focusable.
6. Assigning and revealing keyboard shortcuts, including guidance on how to avoid problematic conflicts with keyboard commands of assistive technologies, browsers, and operating systems.

Fundamental Keyboard Navigation Conventions

ARIA roles, states, and properties model accessibility behaviors and features shared among GUI components of popular desktop GUIs, including Microsoft Windows, macOS, and GNOME. Similarly, ARIA design patterns borrow user expectations and keyboard conventions from those platforms, consistently incorporating common conventions with the aim of facilitating easy learning and efficient operation of keyboard interfaces across the web.

For a web page to be accessible, all interactive elements must be operable via the keyboard. In addition, consistent application of the common GUI keyboard interface conventions described in the APG patterns is important, especially for assistive technology users. Consider, for example, a screen reader user operating a tree. Just as familiar visual styling helps users discover how to expand a tree branch with a mouse, ARIA attributes give the tree the sound and feel of a tree in a desktop application. So, screen reader users will commonly expect that pressing the right arrow key will expand a collapsed node. Because the screen reader knows the element is a tree, it also has the ability to instruct a novice user how to operate it. Similarly, voice recognition software can implement commands for expanding and collapsing branches because it recognizes the element as a tree and can execute appropriate keyboard commands. All this is only possible if the tree implements the GUI keyboard conventions as described in the Tree View Pattern.

A primary keyboard navigation convention common across all platforms is that the `tab` and `shift` + `tab` keys move focus from one UI component to another while other keys, primarily the arrow keys, move focus inside of components that include multiple focusable elements. The path that the focus follows when pressing the `tab` key is known as the tab sequence or tab ring.

Common examples of UI components that contain multiple focusable elements are radio groups, tablists, menus, and grids. A radio group, for example, contains multiple radio buttons, each of which is focusable. However, only one of the radio buttons is included in the tab sequence. After pressing the `Tab` key moves focus to a radio button in the group, pressing arrow keys moves focus among the radio buttons in the group, and pressing the `Tab` key moves focus out of the radio group to the next element in the tab sequence.

The ARIA specification refers to a discrete UI component that contains multiple focusable elements as a composite (https://w3c.github.io/aria/#composite) widget. The process of controlling focus movement inside a composite is called managing focus. Following are some ARIA design patterns with example implementations that demonstrate focus management:

- Combobox
- Grid

- Listbox
- Menu and Menubar
- Radio Group
- Tabs
- Toolbar
- Treegrid
- Tree View

Discernible and Predictable Keyboard Focus

Work to complete this section is tracked by issue 217. (https://github.com/w3c/aria-practices/issues/217)

When operating with a keyboard, two essentials of a good experience are the abilities to easily discern the location of the keyboard focus and to discover where focus landed after a navigation key has been pressed. The following factors affect to what extent a web page affords users these capabilities.

1. Visibility of the focus indicator: Users need to be able to easily distinguish the keyboard focus indicator from other features of the visual design. Just as a mouse user may move the mouse to help find the mouse pointer, a keyboard user may press a navigation key to watch for movement. If visual changes in response to focus movement are subtle, many users will lose track of focus and be unable to operate. Authors are advised to rely on the default focus indicators provided by browsers. If overriding the default, consider:
    - something about ... Colors and gradients can disappear in high contrast modes.
    - Users need to be able to easily distinguish between focus and selection as described in Focus VS Selection and the Perception of Dual Focus(⇘ #kbd_focus_vs_selection), especially when a component that contains selected elements does not contain the focus.
    - ... other considerations to be added ...
2. Persistence of focus: It is essential that there is always a component within the user interface that is active (document.activeElement is not null or is not the body element) and that the active element has a visual focus indicator. Authors need to manage events that effect the currently active element so focus remains visible and moves logically. For example, if the user closes a dialog or performs a destructive operation like deleting an item from a list, the active element may be hidden or removed from the DOM. If such events are not managed to set focus on the button that triggered the dialog or on the list item following the deleted item, browsers move focus to the body element, effectively causing a loss of focus within the user interface.
3. Predictability of movement: Usability of a keyboard interface is heavily influenced by how readily users can guess where focus will land after a navigation key is pressed. Some possible approaches to optimizing predictability include:
    - Move focus in a pattern that matches the reading order of the page's language. In left to right languages, for example, create a tab sequence that moves focus left to right and then top to bottom.

- Incorporate all elements of a section of the page in the tab sequence before moving focus to another section. For instance, in a page with multiple columns that has content in a left side bar, center region, and right side bar, build a tab sequence that covers all elements in the left sidebar before focus moves to the first focusable element in the center column.
- When the distance between two consecutive elements in the tab sequence is significant, avoid movement that would be perceived as backward. For example, on a page with a left to right language, a jump from the last element in the bottom right of the main content to the top element in a left-hand sidebar is likely to be less predictable and more difficult to follow, especially for users with a narrow field of view.
- Follow consistent patterns across a site. The keyboard experience is more predictable when similar pages have similar focus movement patterns.
- Do not set initial focus when the page loads except in cases where:
  - The page offers a single, primary function that nearly all users employ immediately after page load.
  - Any given user is likely to use the page often.

Focus VS Selection and the Perception of Dual Focus

Occasionally, it may appear as if two elements on the page have focus at the same time. For example, in a multi-select list box, when an option is selected it may be greyed. Yet, the focus indicator can still be moved to other options, which may also be selected. Similarly, when a user activates a tab in a tablist, the selected state is set on the tab and its visual appearance changes. However, the user can still navigate, moving the focus indicator elsewhere on the page while the tab retains its selected appearance and state.

Focus and selection are quite different. From the keyboard user's perspective, focus is a pointer, like a mouse pointer; it tracks the path of navigation. There is only one point of focus at any time and all operations take place at the point of focus. On the other hand, selection is an operation that can be performed in some widgets, such as list boxes, trees, and tablists. If a widget supports only single selection, then only one item can be selected and very often the selected state will simply follow the focus when focus is moved inside of the widget. That is, in some widgets, moving focus may also perform the select operation. However, if the widget supports multiple selection, then more than one item can be in a selected state, and keys for moving focus do not perform selection. Some multi-select widgets do support key commands that both move focus and change selection, but those keys are different from the normal navigation keys. Finally, when focus leaves a widget that includes a selected element, the selected state persists.

From the developer's perspective, the difference is simple -- the focused element is the active element (document.activeElement). Selected elements are elements that have `aria-selected="true"`.

With respect to focus and the selected state, the most important considerations for designers and developers are:

- The visual focus indicator must always be visible.

- The selected state must be visually distinct from the focus indicator.

Deciding When to Make Selection Automatically Follow Focus

In composite widgets where only one element may be selected, such as a tablist or single-select listbox, moving the focus may also cause the focused element to become the selected element. This is called having selection follow focus. Having selection follow focus is often beneficial to users, but in some circumstances, it is extremely detrimental to accessibility.

For example, in a tablist, the selected state is used to indicate which panel is displayed. So, when selection follows focus in a tablist, moving focus from one tab to another automatically changes which panel is displayed. If the content of panels is present in the DOM, then displaying a new panel is nearly instantaneous. A keyboard user who wishes to display the fourth of six tabs can do so with 3 quick presses of the right arrow. And, a screen reader user who perceives the labels on tabs by navigating through them may efficiently read through the complete list without any latency.

However, if displaying a new panel causes a network request and possibly a page refresh, the effect of having selection automatically follow focus can be devastating to the experience for keyboard and screen reader users. In this case, displaying the fourth tab or reading through the list becomes a tedious and time-consuming task as the user experiences significant latency with each movement of focus. Further, if displaying a new tab refreshes the page, then the user not only has to wait for the new page to load but also return focus to the tab list.

When selection does not follow focus, the user changes which element is selected by pressing the `Enter` or `Space` key.

Keyboard Navigation Between Components (The Tab Sequence)

As explained in section Fundamental Keyboard Navigation Conventions(⇘ #kbd_generalnav), all interactive UI components need to be reachable via the keyboard. This is best achieved by either including them in the tab sequence or by making them accessible from a component that is in the tab sequence, e.g., as part of a composite component. This section addresses building and managing the tab sequence, and subsequent sections cover making focusable elements that are contained within components keyboard accessible.

The HTML `tabindex` (https://html.spec.whatwg.org/multipage/interaction.html#the-tabindex-attribute) and SVG2 `tabindex` (https://www.w3.org/TR/SVG2/struct.html#tabindexattribute) attributes can be used to add and remove elements from the tab sequence. The value of `tabindex` can also influence the order of the tab sequence, although authors are strongly advised not to use `tabindex` for that purpose.

In HTML, the default tab sequence of a web page includes only links and HTML form elements, except In macOS, where it includes only form elements. macOS system preferences include a keyboard

setting that enables the tab key to move focus to all focusable elements.

The default order of elements in the tab sequence is the order of elements in the DOM. The DOM order also determines screen reader reading order. It is important to keep the keyboard tab sequence and the screen reader reading order aligned, logical, and predictable as described in Discernible and Predictable Keyboard Focus(↘ #kbd_focus_discernable_predictable). The most robust method of manipulating the order of the tab sequence while also maintaining alignment with the reading order that is currently available in all browsers is rearranging elements in the DOM.

The values of the `tabindex` attribute have the following effects.

`tabindex` is not present or does not have a valid value
: The element has its default focus behavior. In HTML, only form controls and anchors with an HREF attribute are included in the tab sequence.

`tabindex="0"`
: The element is included in the tab sequence based on its position in the DOM.

`tabindex="-1"`
: The element is not included in the tab sequence but is focusable with element.focus().

`tabindex="X"` where `X` is an integer in the range `1 <= X <= 32767`
: Authors are strongly advised NOT to use these values. The element is placed in the tab sequence based on the value of `tabindex`. Elements with a `tabindex` value of 0 and elements that are focusable by default will be in the sequence after elements with a `tabindex` value of `1` or greater.

Keyboard Navigation Inside Components

As described in section Fundamental Keyboard Navigation Conventions(↘ #kbd_generalnav), the tab sequence should include only one focusable element of a composite UI component. Once a composite contains focus, keys other than `Tab` and `Shift` + `Tab` enable the user to move focus among its focusable elements. Authors are free to choose which keys move focus inside of a composite, but they are strongly advised to use the same key bindings as similar components in common GUI operating systems as demonstrated in APG Patterns.

The convention for where focus lands in a composite when it receives focus as a result of a `Tab` key event depends on the type of composite. It is typically one of the following.

- The element that had focus the last time the composite contained focus. Or, if the composite has not yet contained the focus, the first element. Widgets that usually employ this pattern include grid and tree grid.
- The selected element. Or, if there is no selected element, the first element. Widgets where this pattern is commonly implemented include radio groups, tabs, list boxes, and trees. Note: For radio groups, this pattern is referring to the checked radio button; the selected state is not supported for radio buttons.
- The first element. Components that typically follow this pattern include menubars and toolbars.

The following sections explain two strategies for managing focus inside composite elements: creating a roving `tabindex` and using the `aria-activedescendant` property.

Managing Focus Within Components Using a Roving `tabindex`

When using roving `tabindex` to manage focus in a composite UI component, the element that is to be included in the tab sequence has `tabindex="0"` and all other focusable elements contained in the composite have `tabindex="-1"`. The algorithm for the roving `tabindex` strategy is as follows.

- When the component container is loaded or created, set `tabindex="0"` on the element that will initially be included in the tab sequence and set `tabindex="-1"` on all other focusable elements it contains.
- When the component contains focus and the user presses a navigation key that moves focus within the component, such as an arrow key:
  - set `tabindex="-1"` on the element that has `tabindex="0"`.
  - Set `tabindex="0"` on the element that will become focused as a result of the key event.
  - Set focus, `element.focus()`, on the element that has `tabindex="0"`.
- If the design calls for a specific element to be focused the next time the user moves focus into the composite with `Tab` or `Shift` + `Tab`, check if that target element has `tabindex="0"` when the composite loses focus. If it does not, set `tabindex="0"` on the target element and set `tabindex="-1"` on the element that previously had `tabindex="0"`.

One benefit of using roving `tabindex` rather than `aria-activedescendant` to manage focus is that the user agent will scroll the newly focused element into view.

Managing Focus in Composites Using `aria-activedescendant`

If a component container has an ARIA role that supports the aria-activedescendant (https://w3c.github.io/aria/#aria-activedescendant) property, it is not necessary to manipulate the `tabindex` attribute and move DOM focus among focusable elements within the container. Instead, only the container element needs to be included in the tab sequence. When the container has DOM focus, the value of `aria-activedescendant` on the container tells assistive technologies which element is active within the widget. Assistive technologies will consider the element referred to as active to be the focused element even though DOM focus is on the element that has the `aria-activedescendant` property. And, when the value of `aria-activedescendant` is changed, assistive technologies will receive focus change events equivalent to those received when DOM focus actually moves.

The steps for using the `aria-activedescendant` method of managing focus are as follows.

- When the container element that has a role that supports `aria-activedescendant` is loaded or created, ensure that:
  - The container element is included in the tab sequence as described in Keyboard Navigation Between Components(↘ #kbd_general_between) or is a focusable element of a composite that implements a roving `tabindex`(↘ #kbd_roving_tabindex).
  - It has `aria-activedescendant="IDREF"` where `IDREF` is the `id` of the element within the container that should be identified as active when the widget receives focus. The referenced element needs to meet the DOM relationship requirements described below.

- When the container element receives DOM focus, draw a visual focus indicator on the active element and ensure the active element is scrolled into view.
- When the composite widget contains focus and the user presses a navigation key that moves focus within the widget, such as an arrow key:
  - Change the value of `aria-activedescendant` on the container to refer to the element that should be reported to assistive technologies as active.
  - Move the visual focus indicator and, if necessary, scrolled the active element into view.
- If the design calls for a specific element to be focused the next time a user moves focus into the composite with `Tab` or `Shift+Tab`, check if `aria-activedescendant` is referring to that target element when the container loses focus. If it is not, set `aria-activedescendant` to refer to the target element.

The specification for `aria-activedescendant` (https://w3c.github.io/aria/#aria-activedescendant) places important restrictions on the DOM relationship between the focused element that has the `aria-activedescendant` attribute and the element referenced as active by the value of the attribute. One of the following three conditions must be met.

1. The element referenced as active is a DOM descendant of the focused referencing element.
2. The focused referencing element has a value specified for the aria-owns (https://w3c.github.io/aria/#aria-owns) property that includes the ID of the element referenced as active.
3. The focused referencing element has role of combobox (https://w3c.github.io/aria/#combobox), textbox (https://w3c.github.io/aria/#textbox), or searchbox (https://w3c.github.io/aria/#searchbox) and has aria-controls (https://w3c.github.io/aria/#aria-controls) property referring to an element with a role that supports `aria-activedescendant` and either:
   1. The element referenced as active is a descendant of the controlled element.
   2. The controlled element has a value specified for the aria-owns (https://w3c.github.io/aria/#aria-owns) property that includes the ID of the element referenced as active.

> Note
>
> This guidance focuses specifically on keyboard interaction. However, authors also need to consider pointer interactions, such as mouse clicks and touchscreen taps. When a component is clicked/tapped, authors should take the same steps to set the correct `tabindex` or `aria-activedescendant` for the element, in the same way that they would for keyboard navigation. Otherwise, this could lead to a confusing experience for users that switch between pointer and keyboard navigation, as it will lead to a mismatch.

Focusability of disabled controls

Browsers remove HTML input elements with the `disabled` attribute from the tab sequence. However, there are some contexts where it is useful for an element to convey a disabled state while remaining focusable, especially inside of composite widgets. This can be accomplished by applying the state `aria-disabled="true"`. For example, as demonstrated in the menu and menubar pattern, disabled items are focusable when navigating through a menu with the arrow keys.

Removing focusability from disabled elements can offer users both advantages and disadvantages. Allowing keyboard users to skip disabled elements usually reduces the number of key presses required to complete a task. However, screen reader users are far less likely to discover disabled elements that are not focusable because moving focus is one of their primary methods of discovery.

Authors are encouraged to adopt consistent pattern-based conventions for the focusability of disabled elements. The examples in this guide adopt the following conventions, which both reflect common practice and attempt to balance competing concerns.

1. When users can reasonably infer the presence of a disabled element from nearby focusable elements, it is removed from the keyboard focus order using the HTML `disabled` attribute. For example:
    - In the “Scrollable Search Results” grid example, when the grid is showing the first page and the “Next” button receives focus, users can infer that the “Previous” button is disabled.
    - A toolbar with buttons for moving, removing, and adding items in a list includes buttons for "Up", "Down", "Add", and "Remove". The "Up" button is disabled and its focusability is removed when the first item in the list is selected. Given the presence of the "Down" button, discoverability of the "Up" button is not a concern.
2. When a disabled element *does* need to remain discoverable, `aria-disabled="true"` is applied so that it will remain focusable. For example:
    - The “Copy”, “Cut”, and “Paste” buttons in the Toolbar. The discoverability of these features relies on their focusability even when they are not immediately applicable (i.e., when no text is selected in the editor and/or when the clipboard is empty).
    - Options in a Listbox
    - Menu items in a Menu or menu bar
    - Tab elements in a set of Tabs
    - Tree items in a Tree View

One design technique for mitigating the impact of including disabled elements in the path of keyboard focus is employing appropriate keyboard shortcuts as described in Keyboard Shortcuts(↘ #kbd_shortcuts).

Key Assignment Conventions for Common Functions

The following key assignments can be used in any context where their conventionally associated functions are appropriate. While the assignments associated with Windows and Linux platforms can be implemented and used in browsers running in macOS, replacing them with macOS assignments in browsers running on a macOS device can make the keyboard interface more discoverable and intuitive for those users. In some cases, it may also help avoid system or browser keyboard conflicts.

Function	Windows/Linux Key	macOS Key
open context menu	`Shift` + `F10`	
Copy to clipboard	`Control` + `C`	`Command` + `C`
Paste from clipboard	`Control` + `V`	`Command` + `V`
Cut to clipboard	`Control` + `X`	`Command` + `X`
undo last action	`Control` + `Z`	`Command` + `Z`
Redo action	`Control` + `Y`	`Command` + `Shift` + `Z`

Keyboard Shortcuts

When effectively designed, keyboard shortcuts that focus an element, activate a widget, or both can dramatically enhance usability of frequently used features of a page or site. This section addresses some of the keyboard shortcut design and implementation factors that most impact their effectiveness, including:

1. Understanding how keyboard shortcuts augment a keyboard interface and whether to make a particular shortcut move focus, perform a function, or both.
2. Making key assignments and avoiding assignment conflicts with assistive technologies, browsers, and operating systems.
3. Exposing and documenting key assignments.

Designing the Scope and Behavior of Keyboard Shortcuts

This section explains the following factors when determining which elements and features to assign keyboard shortcuts and what behavior to give each shortcut:

1. Ensuring discovery through navigation; keyboard shortcuts enhance, not replace, standard keyboard access.
2. Effectively choosing from among the following behaviors:
    1. Navigation: Moving focus to an element.
    2. Activation: Performing an operation associated with an element that does not have focus and might not be visible.
    3. Navigation and activation: Both moving focus to an element and activating it.
3. Balancing efficiency and cognitive load: lack of a shortcut can reduce efficiency while too many shortcuts can increase cognitive load and clutter the experience.

Ensure Basic Access Via Navigation

Before assigning keyboard shortcuts, it is essential to ensure the features and functions to which shortcuts may be assigned are keyboard accessible without a keyboard shortcut. In other words, all elements that could be targets for keyboard shortcuts need to be focusable via the keyboard using the methods described in:

- Keyboard Navigation Between Components(⇘ #kbd_general_between)
- Keyboard Navigation Inside Components(⇘ #kbd_general_within)

Do not use keyboard shortcuts as a substitute for access via navigation. This is essential to full keyboard access because:

1. The primary means of making functions and their shortcuts discoverable is by making the target elements focusable and revealing key assignments on the element itself.
2. If people who rely on the keyboard have to read documentation to learn which keys are required to use an interface, the interface may technically meet some accessibility standards but in practice is only accessible to the small subset of them who have the knowledge that such documentation exists, have the extra time available, and the ability to retain the necessary information.
3. Not all devices that depend on keyboard interfaces can support keyboard shortcuts.

Choose Appropriate Shortcut Behavior

The following conventions may help identify the most advantageous behavior for a keyboard shortcut.

- Move focus when the primary objective is to make navigation more efficient, e.g., reduce the number of times the user must press `Tab` or the arrow keys. This behavior is commonly expected when assigning a shortcut to a text box, toolbar, or composite, such as a listbox, tree, grid, or menubar. This behavior is also useful for moving focus to a section of a page, such as the main content or a complementary landmark section.
- Activate an element without moving focus when the target context of the function is the context that contains the focus. This behavior is most common for command buttons and for functions associated with elements that are not visible, such as a "Save" option that is accessible via a menu. For example, if the focus is on an option in a listbox and a toolbar contains buttons for moving and removing options, it is most beneficial to keep focus in the listbox when the user presses a key shortcut for one of the buttons in the toolbar. This behavior can be particularly important for screen reader users because it provides confirmation of the action performed and makes performing multiple commands more efficient. For instance, when a screen reader user presses the shortcut for the "Up" button, the user will be able to hear the new position of the option in the list since it still has the focus. Similarly, when the user presses the shortcut for deleting an option, the user can hear the next option in the list and immediately decide whether to press the delete shortcut again.
- Move focus and activate when the target of the shortcut has a single function and the context of that function is the same as the target. This behavior is typical when a shortcut is assigned to a button that opens a menu or dialog, to a checkbox, or to a navigation link or button.

Choose Where to Add Shortcuts

Work to draft content for this section is tracked in issue 219. (https://github.com/w3c/aria-practices/issues/219)

The first goal when designing a keyboard interface is simple, efficient, and intuitive operation with only basic keyboard navigation support. If basic operation of a keyboard interface is inefficient, attempting to compensate for fundamental design issues, such as suboptimal layout or command structure, by implementing keyboard shortcuts will not likely reduce user frustration. The practical implication of this is that, in most well-designed user interfaces, the percentage of functionality that needs to be accessible via a keyboard shortcut in order to create optimal usability is not very high. In many simple user interfaces, keyboard shortcuts can be entirely superfluous. And, in user interfaces with too many keyboard shortcuts, the excess shortcuts create cognitive load that make the most useful ones more difficult to remember.

Consider the following when deciding where to assign keyboard shortcuts:

1. To be written.

Assigning Keyboard Shortcuts

When choosing the keys to assign to a shortcut, there are many factors to consider.

- Making the shortcut easy to learn and remember by using a mnemonic (e.g., `Control` + `S` for "Save") or following a logical or spacial pattern.
- Localizing the interface, including for differences in which keys are available and how they behave and for language considerations that could impact mnemonics.
- Avoiding and managing conflicts with key assignments used by an assistive technology, the browser, or the operating system.

Methods for designing a key shortcut scheme that supports learning and memory is beyond the scope of this guide. Unless the key shortcut scheme is extensive, it is likely sufficient to mimic concepts that are familiar from common desktop software, such as browsers. Similarly, while localization is important, describing how to address it is left to other resources that specialize in that topic.

The remainder of this section provides guidance balancing requirements and concerns related to key assignment conflicts. It is typically ideal if key assignments do not conflict with keys that are assigned to functions in the user's operating system, browser, or assistive technology. Conflicts can block efficient access to functions that are essential to the user, and a perfect storm of conflicts can trap a user. At the same time, there are some circumstances where intentional conflicts are useful. And, given the vast array of operating system, browser, and assistive technology keys, it is almost impossible to be certain conflicts do not exist. So it is also important to employ strategies that mitigate the impact of conflicts whether they are intentional or unknown.

> Note

> In the following sections, `meta` key refers to the `Windows` key on Windows-compatible keyboards and the `Command` key on MacOS-compatible keyboards.

Operating System Key Conflicts

It is essential to avoid conflicts with keys that perform system level functions, such as application and window management and display and sound control. In general, this can be achieved by refraining from the following types of assignments.

1. Any modifier keys + any of `Tab`, `Enter`, `Space`, or `Escape`.
2. `Meta` key + any other single key (there are exceptions, but they can be risky as these keys can change across versions of operating systems).
3. `Alt` + a function key.

In addition, there are some important application level features that most applications, including browsers, generally support. These include:

1. Zoom
2. Copy/Paste
3. ... to be continued ...

Assistive Technology Key Conflicts

Even though assistive technologies have collectively taken thousands of key assignments, avoiding conflicts is relatively easy. This is because assistive technologies have had to develop key assignment schemes that avoid conflicts with both operating systems and applications. They do this by hijacking specific keys as modifiers that uniquely define their key commands. For example, many assistive technologies use the `Caps Lock` key as a modifier.

Deflect assistive technology key conflicts by steering clear of the following types of assignments.

1. `Caps Lock` + any other combination of keys.
2. `Insert` + any combination of other keys.
3. `Scroll Lock` + any combination of other keys.
4. macOS only: `Control` + `Option` + any combination of other keys.

Browser Key Conflicts

While there is considerable similarity among browser keyboard schemes, the patterns within the schemes are less homogenous. Consequently, it is more difficult to avoid conflicts with browser key assignments. While the impact of conflicts is sometimes mitigated by the availability of two paths to nearly every function -- keyboard accessible menus and keyboard shortcuts, avoiding conflicts with shortcuts to heavily used functions is nonetheless important. Pay special attention to avoiding conflicts with shortcuts to:

1. Address or location bar

2. Notification bar
3. Page refresh
4. Bookmark and history functions
5. Find functions

Intentional Key Conflicts

While avoiding key conflicts is usually desirable, there are circumstances where intentionally conflicting with a browser function is acceptable or even desirable. This can occur when the following combination of conditions arises:

- A web application has a frequently used function that is similar to a browser function.
- Users will often want to execute the web application function.
- Users will rarely execute the browser function.
- There is an efficient, alternative path to the browser function.

For example, consider a save function that is available when the focus is in an editor. Most browsers use … to be continued …

W3C Web Accessibility Initiative (WAI) (https://www.w3.org/WAI/)

Strategies, standards, resources to make the Web accessible to people with disabilities

Get News in Email (https://www.w3.org/WAI/news/subscribe/)

LinkedIn (https://www.linkedin.com/company/w3c-wai/) Mastodon (https://w3c.social/@wai)

YouTube (https://www.youtube.com/channel/UCU6ljj3m1fglIPjSjs2DpRA/playlists/)

Home (https://www.w3.org/WAI/) Contact (https://www.w3.org/WAI/about/contacting/)

Site map (https://www.w3.org/WAI/sitemap/) Support WAI (https://www.w3.org/WAI/about/support/)

News (https://www.w3.org/WAI/news/)

Accessibility statement (https://www.w3.org/WAI/about/accessibility-statement/)

All Translations (https://www.w3.org/WAI/translations/) Resources for roles (https://www.w3.org/WAI/roles/)

Copyright © 2026 World Wide Web Consortium (https://www.w3.org/).
W3C® liability (https://www.w3.org/policies/#disclaimers), trademark (https://www.w3.org/policies/#trademarks) and permissive license (https://www.w3.org/copyright/software-license/) rules apply unless otherwise noted. See Permission to Use WAI Material (https://www.w3.org/WAI/about/using-wai-material/).